

PROJECT PLAN

Dineo Nkuna-Project manager

Ditebogo Nkwana- Lead software developer

Thembelihle Ngobeni- Data analyst

Melepe Molomotsi- Database admin

Table of Contents

INTRODUCTION & PROJECT VISION 3

PROJECT SCOPE and Responsibility matrix 3

1. Scope Statement 3

2. Project Objectives 3

3. In-Scope Features 4

4. Technical Scope 4

5. Project Deliverables 4

6. Project Constraints 5

7. Out of Scope 5

8. Scope Completion Criteria 5

Responsibility matrix 6

1. REQUIREMENTS ANALYSIS 6

1.1 Functional Requirements..... 6

1.2 Non-Functional Requirements (Quality Attributes) 8

1.3 Project Constraints 8

2. DEVOPS LIFECYCLE (CI/CD PIPELINE) 8

2.1 Planning and Version Control10

2.2 Build and Test Automation10

2.3 Deployment and Release Management10

3. SYSTEM DESIGN DOCUMENT (SDD)12

3.1 High-Level System Architecture12

3.2 Technology Stack Justification.....13

3.3 Database Schema (Firestore Collections)14

3.4 Critical API Endpoints (Node.js Middleware).....17

3.5 User Interface and Wireframes (Description)18

4. Security Measures and Hardening.....18

Project plan..... 0

..... 2

Deliverables 4

INTRODUCTION & PROJECT VISION

The DMD Food Bank Management System is a digital ecosystem designed to bridge the critical gap between food surplus, volunteer workforce with homeless shelters and orphanages.

The core mission is to eliminate food insecurity by providing a transparent, real-time logistics platform. The system allows individual donors and organizations to see exactly where their contributions are needed on any given date. It facilitates volunteer sign-ups for cooking shifts, enables shelters to apply for assistance and provides donors as well as the public with tangible impact reports (showing photos and meal counts). By centralizing these operations into a single Android and Web application, DMD aims to increase donation efficiency and decrease hunger .

PROJECT SCOPE and Responsibility matrix

1. Scope Statement

The scope of the **DMD Food Bank Management System** is to design, develop, test, and deploy a digital platform that improves the management of food donations, volunteer activities, shelter/orphanage requests, and food-bank operations. The system aims to connect donors, volunteers, administrators, shelters, orphanages, NGOs, and corporate partners through a centralized platform. The project will provide both a **web application and an Android mobile application**, allowing users to access relevant food-bank services digitally. This supports the project's overall vision of improving donation efficiency and helping reduce food insecurity.

2. Project Objectives

The project will:

- Develop a secure and user-friendly DMD Food Bank platform.
- Allow donors to make monetary and physical food donations.
- Display real-time information about food items required by shelters and orphanages.
- Allow volunteers to view, book, reschedule, and cancel available cooking shifts.
- Allow shelters and orphanages to apply for assistance and monitor their application status.
- Provide donors with information about the impact of their contributions, including meals funded and photo updates.
- Allow NGOs and corporate sponsors to submit collaboration proposals.
- Provide administrators with tools to manage donations, applications, impact stories, and other system information.

3. In-Scope Features

The project includes the development of the following major components:

Donor Portal: Donors will be able to create accounts, make monetary donations using the integrated payment system, identify drop-off options for physical food donations, view current shelter needs, and track the impact of their contributions.

Volunteer Management: Volunteers will be provided with a scheduling system where they can browse available cooking shifts and book, reschedule, or cancel their participation.

Shelter and Orphanage Portal: Shelters and orphanages will be able to submit applications for assistance, upload relevant registration information, provide capacity details, and monitor whether applications are pending, under review, approved, or rejected.

Administration System: Administrators will have access to a CMS/dashboard for allocating donations, reviewing shelter applications, publishing impact stories, and updating information displayed by the system.

Impact Tracking: The system will maintain impact information such as photographs, stories, donation references, and the number of meals served. This information will be used to provide donors with feedback regarding the impact of their contributions.

Collaboration Management: NGOs, corporate sponsors, and other organisations will be able to submit partnership proposals through the platform.

4. Technical Scope

The system will use a serverless-oriented architecture. The web frontend will use **React.js**, while the Android application will use **React Native with Expo**. Firebase Firestore will provide the database, Firebase Authentication will manage user authentication, and Firebase Cloud Storage will store media such as impact photographs.

The project also includes secure payment functionality, web hosting, Android distribution, automated testing, deployment, monitoring, and security controls. The project plan specifies Stripe for payments, Vercel for web hosting, and Google Play Store distribution for the Android application.

5. Project Deliverables

The main deliverables of the project will include the completed **DMD Food Bank web application, Android mobile application, database and authentication system, donation and payment functionality, real-time needs board, volunteer scheduling system, shelter/orphanage application system, administrator CMS, impact tracking functionality, testing and bug fixes, deployment, and project documentation.**

The project plan's Gantt chart on page 15 also identifies database and authentication, donation portal and needs board, volunteer scheduler, shelter portal and admin CMS, impact tracker and integration, testing and bug fixes, deployment, user acceptance testing, maintenance, and documentation as major project activities.

6. Project Constraints

The project is subject to three major constraints. First, it must operate at **zero monthly operational cost**, making use of available free-tier services. Second, the mobile application is limited to **Android**, beginning with Android 7.0/API Level 24 and above. Third, development is limited to a **13-week timeframe**, requiring the team to follow structured Agile sprint cycles.

7. Out of Scope

To keep the project achievable within the defined timeframe and constraints, the following are outside the current project scope:

- Development of an android application.
- Management of the physical transportation of donated food.
- Physical preparation or cooking of food.
- Construction or management of shelters and community kitchens.
- Direct management of third-party banking/payment infrastructure.
- Features that are not included in the approved project requirements.
- Paid infrastructure or services that would conflict with the project's zero-monthly-operational-cost constraint.

8. Scope Completion Criteria

The project scope will be considered complete when the major web and Android features are implemented, the database and authentication functionality are operational, donation and volunteer functions work as specified, administrators can manage the required information, testing and bug fixing have been completed, and the final Android application is prepared for Google Play Store submission. The project plan allocates the final sprint to **UI polishing, bug fixes, and Google Play Store submission**.

Responsibility matrix



RESPONSIBILITY MATRIX (RACI)

DMD FOOD BANK MANAGEMENT SYSTEM

People | Food | Community | Impact

Together
We End
Hunger

Project Activity / Deliverable	Dineo (Project Manager)	Nkwana (Lead Software Developer)	Lithle (Data Analyst)	Mogomotse (Database Admin)
Project planning & coordination	A/R	C	C	I
Requirements analysis	A	C	R	C
Database schema design	I	C	C	A/R
Firebase authentication	I	R	I	A
User profile setup	I	A/R	C	C
Donation portal development	I	A/R	C	C
Stripe payment integration	I	A/R	I	C
Real-Time Needs Board	I	R	A	C
Volunteer shift scheduler	I	A/R	C	C
Shelter/Orphanage application portal	C	R	A	C
Admin CMS development	C	A/R	C	C
Database management	I	C	C	A/R
Impact tracker & photo uploads	I	R	A	C
Data evaluation & reporting	C	I	A/R	C
System testing	A	R	R	C
Bug fixing	I	A/R	C	C
User Acceptance Testing (UAT)	A	C	R	I
System security	I	R	C	A/R
Deployment	A	R	I	C
Documentation & final reporting	A/R	C	R	C
Final project submission	A/R	C	C	I

Key:

- R = Responsible**
(Does the work)
- A = Accountable**
(Owns the outcome)
- C = Consulted**
(Provides input)
- I = Informed**
(Kept up to date)

Empowering Communities
Through Food Support

1. REQUIREMENTS ANALYSIS

1.1 Functional Requirements

The system is divided into four distinct user portals, each with specific functionalities.

A. Donor-Facing Features (Android & Web)

1. Multi-Channel Donation Portal. Users can make monetary donations through secure Instant EFT integration or confirmed and secure drop off zones for physical food items.
2. Real-Time Needs Board. An interactive map and list view displaying which shelters/ orphanages require specific food items (for example "Rice, Canned Vegetables") specifically for today's date. This board updates in real-time as needs are fulfilled.
3. Personal Impact Dashboard. After logging in, donors can view a personalized timeline showing the total meals their specific contributions have funded, accompanied by photo

updates and thank-you notes from recipient shelters.

B. Volunteer-Facing Features (Android & Web)

4. Shift Scheduling Module. Volunteers can browse a calendar of available cooking shifts at partner community kitchens. They can book, reschedule, or cancel shifts up to 24 hours in advance.

5. Shelter/Orphanage Application Portal. A comprehensive digital application form for new shelters to register. This includes uploading registration certificates and providing capacity details. Applicants can track their application status (Pending / Under Review / Approved / Rejected).
6. Collaboration Application Form. Dedicated portal for NGOs and corporate sponsors to apply for strategic partnerships, allowing them to propose bulk sponsorship deals.

C. Administrator-Facing Features (Web Admin Panel)

7. Content Management System (CMS). Admins possess a master dashboard to manually allocate donations, approve/reject shelter applications, publish impact stories, and dynamically update contact information and social media links without needing to alter the source code.

1.2 Non-Functional Requirements (Quality Attributes)

1. (Performance): The "Real-Time Needs Board" must render and populate data within 2 seconds on a 4G network connection.
2. Scalability): The backend architecture must support up to 1,000 concurrent users during high-traffic donation drives or promotional events.
3. (Security & Compliance): All payment transactions must be PCI-DSS compliant (handled via Stripe redirects). User passwords must be securely hashed using bcrypt.
4. (Accessibility): The web portal must meet WCAG 2.1 AA standards (proper aria-labels, color contrast ratios, and keyboard navigability).
5. (Reliability): The system guarantees 99.5% uptime for the donation portal, with automated Firebase alerts to notify the development team of any service disruptions.

1.3 Project Constraints

- Budget: The system must be deployable at zero monthly operational cost, utilizing free-tier services (Firebase Spark Plan, Vercel Hobby Tier).
- Platform: The mobile application is exclusively developed for Android (API Level 24 / Android 7.0 Nougat and above) to cover approximately 15% of active devices.
- Timeline: Development is strictly capped at 12 weeks, requiring strict adherence to Agile Sprint cycles.

2. DEVOPSLIFECYCLE (CI/CD PIPELINE)

We have adopted a Continuous Integration and Continuous Deployment (CI/CD) methodology to ensure rapid, stable delivery over our 12-week horizon. The lifecycle is

broken down into bi-weekly Sprints.

2.1 Planning and Version Control

- Project Management: We utilize GitHub Projects with a Kanban board categorized into: Backlog, To-Do, In Progress, Code Review, and Done.
- Branching Strategy: We implement the Git Feature-Branch Workflow.
 - main branch holds production-ready code.
 - develop branch serves as the integration staging area.
 - feature/* branches are created for each specific task (e.g., feature/donation-module).

2.2 Build and Test Automation

- Continuous Integration (CI): Using GitHub Actions, we automatically run linters (ESLint) and unit tests (Jest for React Native) on every pull request. Merges are blocked unless all tests pass.
- Testing Tracks:
 - Alpha Testing: Internal builds distributed to the team via Firebase App Distribution.
 - Beta Testing: We will utilize the Google Play Console's Internal Testing Track, inviting up to 50 external testers via their Gmail addresses to install the app directly from the Play Store with a private link.
 - Automated Builds: For Android, we use Expo EAS Build to generate signed Android App Bundles (AAB) without requiring local Android SDK configuration on every developer's machine.

2.3 Deployment and Release Management

- Web Deployment: The main branch is automatically deployed to Vercel via GitHub Actions upon merge.
- Android Deployment: Upon merging to main, the CI pipeline automatically builds a new AAB and uploads it to the Google Play Console's Internal/Closed Testing track.
- Production Release: Given that Google Play reviews typically take 2 to 24 hours (significantly faster than Apple), we will submit the final build for Production review at the start of Sprint C, ensuring ample buffer time for approval.
- Monitoring: We integrate Firebase Crashlytics for real-time Android crash reporting and Google Analytics for web user behaviour. Firebase Alerts are configured to email the team if database read/write operations exceed 80% of the free-tier quota.

Sprint	Duration	Focus Area
--------	----------	------------

Sprint 1	Weeks 1-2	Database Schema design, Firebase Authentication, and User Profile setup.
Sprint 2	Weeks 3-4	Donation Portal (Stripe integration) and Live Needs Board UI.
Sprint 3	Weeks 5-6	Volunteer Shift Scheduler and Calendar view.
Sprint 4	Weeks 7-8	Shelter/Orphanage Application Forms and Admin CMS (CRUD operations).
Sprint 5	Weeks 9-10	Impact Tracker, Photo Uploads, and End-to-End Integration Testing.
Sprint C	Weeks 11-12	Final UI Polish, Bug Fixes, and Google Play Store Submission.

3. SYSTEMDESIGNDOCUMENT(SDD)

3.1 High-Level System Architecture

The system employs a modern Serverless Architecture to minimize overhead.

- **API Gateway:** While clients primarily interact directly with Firebase SDKs, a lightweight Node.js/Express middleware server is introduced to handle sensitive operations—specifically, generating Stripe Payment Intents and listening for Stripe Webhooks.
- **Data Layer:** All persistent data is stored in Firebase Firestore (NoSQL), user authentication is managed via Firebase Authentication, and media assets (impact photos) are stored in Firebase Cloud Storage.

3.2 Technology Stack Justification

Component	Technology	Justification
Frontend (Web)	React.js	Component-based architecture allows for rapid UI development and reusability.
Mobile (Android)	React Native (Expo)	Allows sharing of ~C0% of code with the Web frontend. Generates a native APK/AAB for Android without requiring Swift/ Kotlin expertise.
Backend/Database	Firebase Firestore	Real-time data synchronization is critical for the "Live Needs Board" (updates instantly when inventory changes).

Authentication	Firebase Auth	Supports Email/ Password, Google, and Facebook logins out-of-the-box.
Payments	Stripe Checkout	Handles PCI compliance entirely on Stripe's servers; we only handle the payment_intent client secret.
Web Hosting	Vercel	Zero-configuration deployment with automatic HTTPS and CDN caching.
App Distribution	Google Play Store	Standard distribution channel for Android; allows staged rollouts.

3.3 Database Schema (Firestore Collections)

We utilize a flexible NoSQL structure to accommodate varying donation types.

Collection Name	Document Fields	Purpose
users	uid (string), email, displayName, role (donor/volunteer/	Centralized repository for all authenticated

	admin/shelter), phone, fcmToken (push notifications)	users.
donations	donorId (reference), amount (number), itemType (cash/ food), shelterId (reference), timestamp (timestamp), status (pending/ completed/failed)	Tracks the financial and physical inflow.
needs_board	shelterId (reference), foodItems[] (array of strings), urgentLevel (high/ medium/low), dateNeededBy (timestamp), isFulfilled (boolean)	Powers the primary dashboard. Admin updates this daily.
volunteer_shifts	kitchenLocation (geopoint), date, startTime, endTime, maxSlots	Manages the cooking schedule and capacity limits.

	(number), currentVolunteers[] (array of userIDs)	
shelter_applica-tions	capacity(number), contactPerson, status (pending/ approved/ rejected), submittedAt	
impact_updates	shelterId (reference), donationId (reference), photoURL (string), storyText, mealsServed (number), timestamp	Feeds the "Impact Dashboard" for donors.
collaborations	orgName, type (corporate/ sponsor/NGO), proposalText, status (pending/ approved/ declined), submittedAt	Tracks partnership proposals.

3.4 Critical API Endpoints (Node.js Middleware)

To secure secret keys, the Android app and Web client will call these specific endpoints hosted on the Node.js server:

Method	Endpoint	Description
POST	/api/create-payment-intent	Accepts donation amount and returns a client_secret to the app to confirm payment on the frontend.
POST	/api/stripe-webhook	Backend-only. Stripe pings this URL upon successful charge; our server then updates the donations collection status to "completed" and increments the shelter's allocation.
GET	/api/needs/current	Fetches today's active needs

		(cached for C0 seconds to reduce Firestore reads).
POST	/api/shelter/apply	Submits a new shelter application (validates registration number format before saving to Firestore).

3.5 User Interface and Wireframes (Description)

We have designed a minimalist, compassionate UI palette using warm oranges and deep greens to evoke trust and nourishment.

- Screen A (Homepage): Features a prominent "Counter" displaying total meals served today, a large "Donate Now" floating action button (FAB), and a carousel of the top 3 urgent shelter requests.
- Screen B (Live Needs Map): Integrates Google Maps SDK for Android showing shelter pins. Clicking a pin opens a bottom sheet listing the specific food items required for that shelter today.
- Screen C (Volunteer Calendar): Displays a monthly grid. Dates with available shifts are highlighted in green. Clicking a date reveals a list of shifts with slots remaining.
- Screen D (Impact Timeline): A reverse-chronological list (like a social media feed) where each post contains a photo of the shelter receiving goods and a "Meals Served" metric linked to the user's specific contribution.

4. Security Measures and Hardening

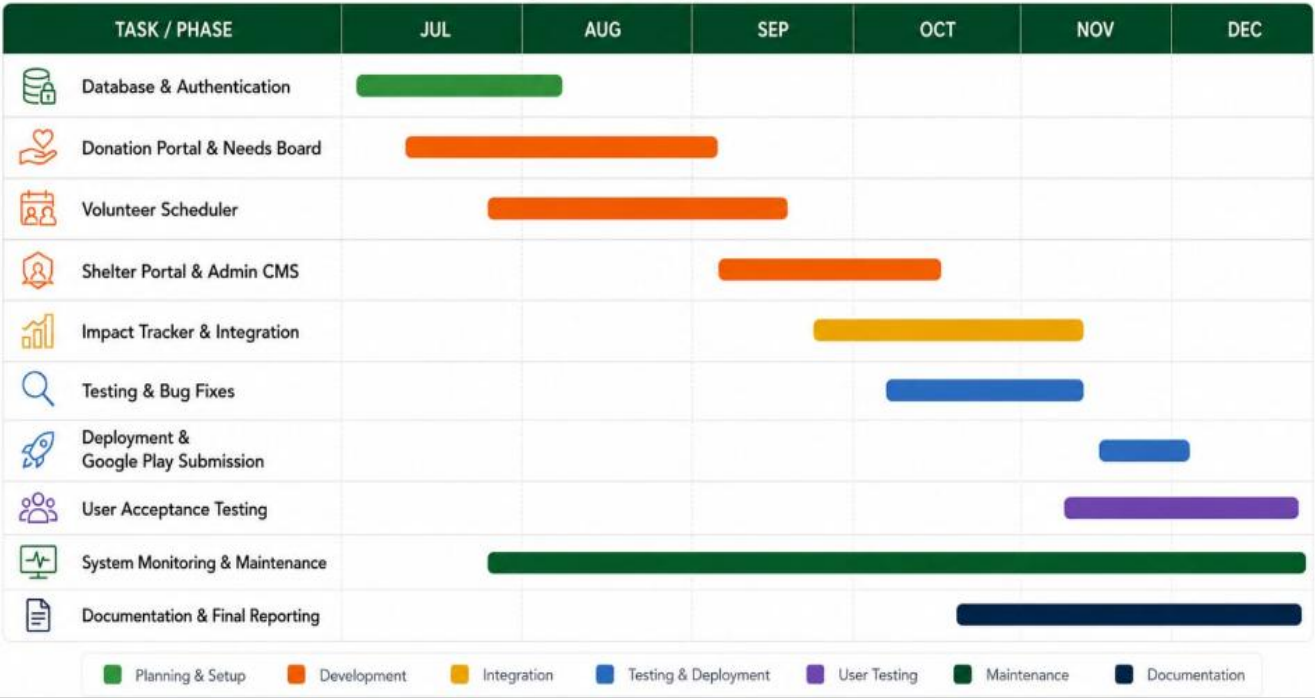
Given the sensitivity of financial transactions and personal data, we employ multiple layers of security:

- Firestore Security Rules: Database access is strictly enforced via Firebase Security Rules. For example:
- match /users/{userId}: allow read, write: if request.auth.uid == userId;
- match /donations/{doc}: allow read: if request.auth != null; allow write: if
 - request.auth.uid == resource.data.donorId;
- Android App Hardening: In the build.gradle file, we enable ProGuard (R8) obfuscation to shrink and obfuscate the code, making reverse-engineering difficult.
- Network Security: The Android app employs a Network Security Configuration XML file to strictly forbid cleartext (HTTP) traffic, ensuring all communication with Firebase and our API occurs over TLS 1.2/1.3 (HTTPS).
- Environment Isolation: All API keys (Firebase, Stripe) are stored in environment variables via .env files, which are explicitly added to .gitignore to prevent them from being exposed in version control.
- Input Sanitization: On the web portal, we utilize DOMPurify to sanitize any HTML input from the CMS before rendering it, preventing Cross-Site Scripting (XSS) attacks.

Project plan and Website screenshots

DMD FOOD BANK MANAGEMENT SYSTEM – GANTT CHART

Project Timeline: July – December





**DMD Food
Bank**



Dineo

Logout

DMD Food Bank

Fighting hunger and food insecurity across South Africa since 2020. Registered NPO.

NPO Reg: 123-456 - PBO Reg: 930 012 345
POPIA compliant - Section 18A tax receipts

f

%

□

w

GET INVOLVED

[Donate money](#)

[Donate food](#)

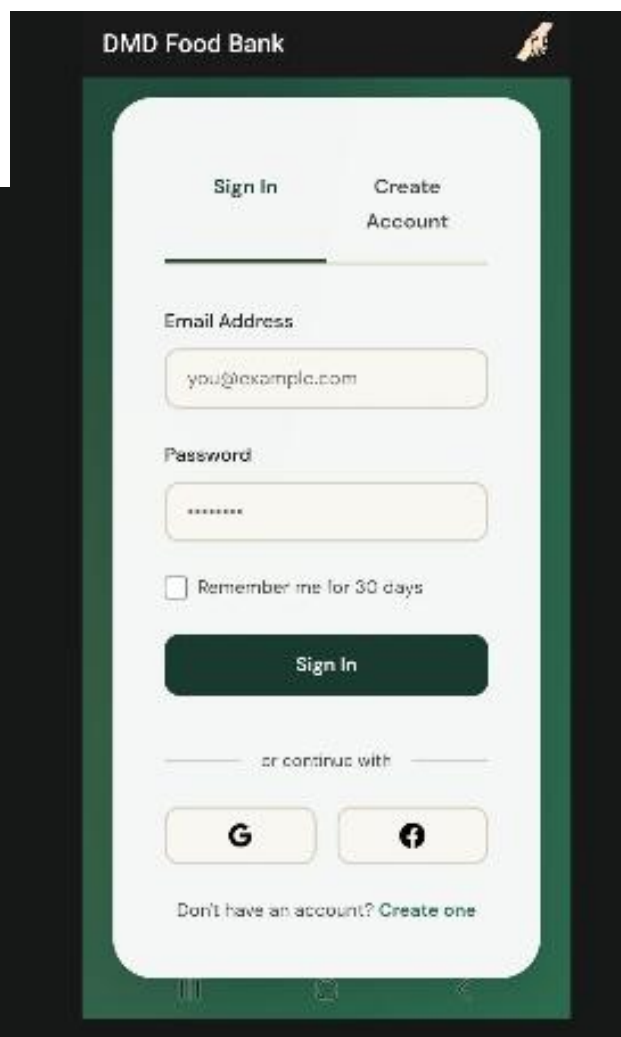
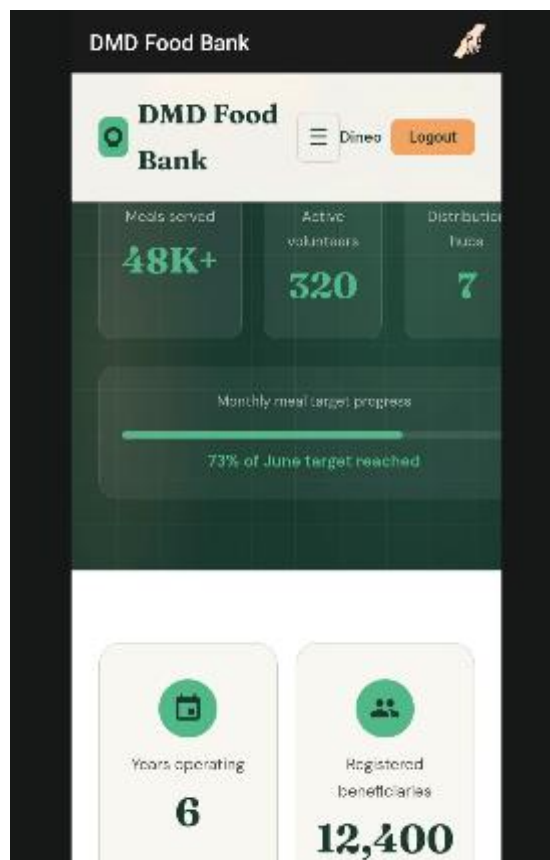
[Corporate partnerships](#)

[Contact us](#)

ABOUT US

[Our story](#)

[Our locations](#)



Deliverables



GitHub Link

<https://github.com/ST10457142/XISD6329-DMDFoodBank>